Project ID: PCD/2425/12

University of Manouba
National School of Computer Science

Report of the Design and Development Project

Subject: R2C-SLAM: Design and implementation of a 2-robot collaborative SLAM system

Authors :

Ms. Meriem Sakka

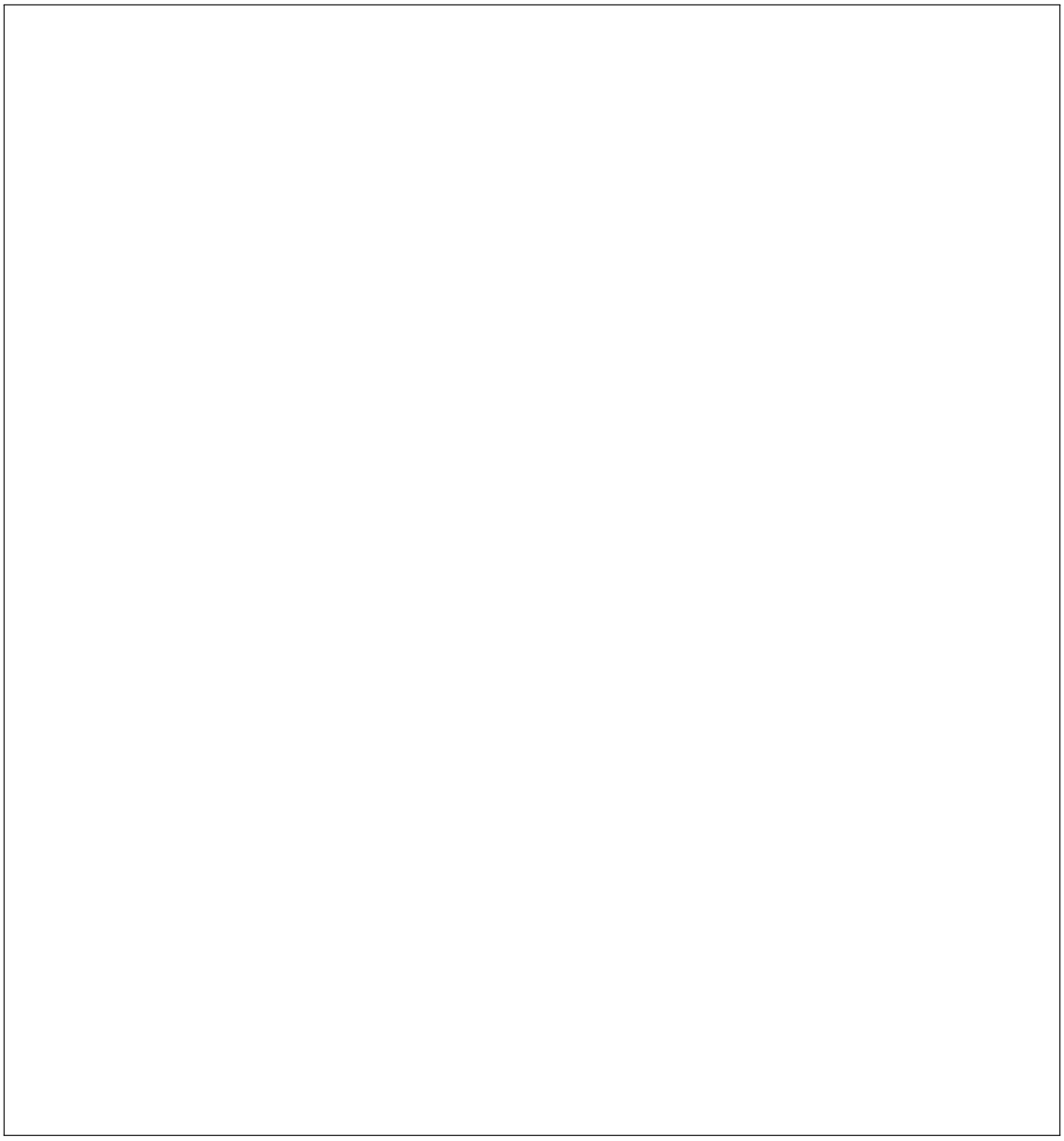
Mr. Oubaid Mahfoudhi Mr. Mohamed Aziz Mensi

Supervisor :

Dr. Chadlia Jerad

Academic Year : 2024/2025

Appreciation and Supervisor's Signature



Abstract— The R2C-SLAM project focuses on designing and implementing a collaborative SLAM (Simultaneous Localization and Mapping) system shared between two mobile robots. The system aims to enhance real-time environment mapping and localization accuracy through inter-robot communication and data sharing. Using low-cost hardware such as Raspberry Pi boards and LIDAR, the project explores how resource-constrained platforms can still achieve reliable SLAM performance. The robots autonomously explore and navigate their surroundings while exchanging mapping and localization data over a robust communication framework based on ROS 2 DDS. Key functionalities include real-time map generation, obstacle avoidance, task synchronization, and visualization via a user interface. The system is tested in simulation using Gazebo and validated in real-world scenarios. This project highlights the feasibility of multi-robot SLAM systems under budget constraints, offering a scalable and efficient solution for collaborative robotics applications.

Keywords: Collaborative SLAM, Multi-Robot Systems, Real-Time Mapping, ROS 2, Embedded Systems, Robot Communication, Autonomous Navigation, Obstacle Avoidance, Map Merging, Gazebo Simulation, Low-Cost Robotics.

Acknowledgements

We would like to express our deepest gratitude to **Dr. Chadlia JERAD**, our supervisor, for her invaluable guidance, constant support, and insightful feedback throughout the entire duration of our PCD project. Her expertise and encouragement were essential to the successful completion of this work.

We also thank the faculty and staff of the National School of Computer Science (ENSI) for providing us with a solid academic foundation and the necessary resources to carry out this project.

Contents

List of Figures

List of Tables

List of Acronyms

SLAM	*Simultaneous Localization and Mapping*
R2C	*2-Robot Collaborative*
DDS	*Data Distribution Service*
C-SLAM	*Collaborative SLAM*
LIDAR	*Light Detection and Ranging*
ROS	*Robot Operating System*
MQTT	*Message Queuing Telemetry Transport*
GUI	*Graphical User Interface*
ORB	*Oriented FAST and Rotated BRIEF*
API	*Application Programming Interface*
UI	*User Interface*
CPU	*Central Processing Unit*
ORB-SLAM3	*Oriented FAST and Rotated BRIEF – SLAM version 3*

Introduction

On 21 October 2024, Gartner Inc., a leading research firm offering insights and advice on technology and business strategy, announced in [1] *Polyfunctional Robots* as one of the top 10 Strategic Technology Trends for 2025. These systems require human-machine and machine-machine collaboration. Many robotic missions depend on accurate localization to navigate and operate effectively within their environments. Autonomous mobile robots rely on SLAM (Simultaneous Localization and Mapping) to navigate and build maps of unknown environments. While traditional SLAM systems involve a single robot, collaborative SLAM enables multiple robots to share data, improving mapping efficiency, accuracy, and robustness.

The R2C-SLAM project aims to implement a collaborative SLAM system for two mobile robots using low-cost, resource-constrained hardware such as Raspberry Pi boards and ultrasonic sensors. Each robot performs SLAM independently while exchanging data through a decentralized communication framework based on ROS 2 and DDS. The robots coordinate in real time to explore, avoid obstacles, and merge their maps. The system is validated through both simulation with Gazebo and real-world testing. This report outlines the system's design, implementation, and evaluation, demonstrating the potential of embedded multi-robot collaboration.

There are five chapters in this thesis, followed by a conclusion. An overview of the ROS 2 framework, a review of relevant work, and the context and motivation for collaborative SLAM are presented in Chapter 1. The system's high-level architectural design, communication model, and functional, non-functional, and domain-specific requirements are presented in Chapter 2. The simulation phase is covered in full in Chapter 3, along with the software stack, intra- and inter-robot SLAM integration, robot selection, and map merging outcomes. The real-world deployment is the main topic of Chapter 4, which covers SLAM execution, hardware and software setup, and actual map merging with robots based on Raspberry Pis. The simulation findings and the real-world deployment are compared in Chapter 5, with an emphasis on the main distinctions, technical difficulties, and lessons discovered. The thesis wraps up by summarizing the results and suggesting possible paths of inquiry for future work.

Chapter 1

Project Overview and Related Work

Introduction

The need for effective autonomous exploration in complex environments has made collaborative simultaneous localization and mapping, or SLAM, a crucial field of robotics research. The perception range, computation power, and lack of redundancy of conventional single-robot SLAM systems are their main drawbacks. By putting in place a system that enables two autonomous mobile robots to work together in real time to create a shared map of an uncharted area.

1.1 Project Overview

1.1.1 Motivation for Collaborative SLAM

Coordination between robots improves accuracy and performance, regardless of whether SLAM is used for navigation, state estimation, or environment mapping. Collaborative SLAM (C-SLAM) extends traditional SLAM to multi-robot systems, allowing teams of robots to work together for faster and more efficient task execution. Rather than running isolated SLAM instances on each robot, C-SLAM algorithms fuse data from all agents to produce a globally consistent map and estimate each robot's state [7]. This shared understanding improves localization and mapping, but also introduces challenges related to data sharing, synchronization, and system complexity inherent in multi-robot coordination.

1.1.2 Objectives of R2C-SLAM

The main objectives of the R2C-SLAM project are as follows:

- Develop a collaborative SLAM system for two mobile robots.

- Enable real-time mapping, localization, and data exchange.
- Ensure system compatibility with low-cost hardware (e.g., Raspberry Pi).
- Validate the system in both simulated (Gazebo) and real-world environments.
- Provide a user interface for real-time visualization and manual control.

These objectives aim to build a robust, scalable, and affordable collaborative robotics platform.

1.1.3 Scope of the Project

This project is limited to indoor exploration using two mobile robots. The system architecture is designed to support future scalability but focuses on two agents for initial validation.

1.2 Related Work on Multi-Robot SLAM

Research in multi-robot SLAM has evolved significantly in recent years due to the growing demand for autonomous multi-agent systems in exploration, surveillance, disaster response, and warehouse automation. Unlike single-robot SLAM systems, multi-robot SLAM presents additional challenges such as inter-robot communication, data association, map merging, and distributed decision-making. Several research projects and frameworks [3] have proposed different strategies to tackle these issues, depending on the level of centralization, hardware capabilities, and application domains.

1.2.1 TurtleBot3 Multi-Robot SLAM

TurtleBot is a low-cost, personal robot kit with open-source software. TurtleBot3 [12] (illustrated in Figure 1.1), created in a partnership between Open Robotics and ROBOTIS, are widely used in academic and research environments due to their compatibility with ROS, the widely used middleware for robotics (introduced in section 1.3), and modular hardware.

Multi-robot SLAM systems using TurtleBot3 typically rely on ROS topics or services to share occupancy grids or transform frames. Projects using Cartographer or GMapping [11] demonstrate basic map merging using shared coordinate transforms and centralized merging nodes. While accessible and replicable, these systems can suffer from synchronization issues, network bandwidth limitations, and single points of failure during centralized data handling.

Figure 1.1: TurtleBot3 Burger

1.2.2 Swarm SLAM Architectures

Swarm-based SLAM approaches are inspired by collective intelligence found in biological systems. These architectures emphasize decentralized control, fault tolerance, and scalability. Robots in a swarm typically use local communication to share partial maps or exploration frontiers. Map merging and global optimization can be distributed, reducing dependence on a central node. Although robust and scalable, swarm SLAM methods often face challenges with global map consistency, loop closure accuracy, and the complexity of consensus algorithms needed to merge partial maps effectively [13].

1.2.3 Comparative Analysis of Collaborative Architectures

In a collaborative system, the choice of the architecture, whether *centralized* or *decentralized*, is important. Figure 1.2 illustrates such architectures. Ultimately, the desired application, available resources, and performance limitations, as well as the consistency of the navigation information, determine whether to use centralized or decentralized C-SLAM.

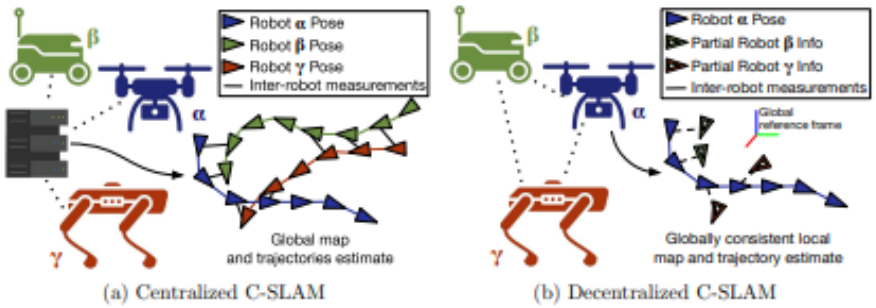


Figure 1.2: Comparison between centralized and decentralized SLAM architectures

Table 1.1 summarizes key differences between centralized, decentralized, and semi-centralized collaborative SLAM architectures based on critical comparison criteria.

Table 1.1: Comparison of Centralized, Decentralized, and Semi-Centralized SLAM Architectures

Criteria	**Centralized SLAM**	**Decentralized SLAM**	**Semi-Centralized SLAM (R2C-SLAM)**
Control	Central server or node	Fully distributed across all agents	Local autonomy + periodic coordination
Fault Tolerance	Low (single point of failure)	High (redundancy among agents)	Medium (partial redundancy, managed failures)
Communication Load	High (continuous streaming to center)	Moderate to low (peer-to-peer or local sharing)	Moderate (periodic data aggregation)
Scalability	Poor (processing limits at center)	High (agents scale independently)	Good (controlled sharing, scalable messaging)
Consistency	High (if no failures)	Challenging (requires consensus mechanisms)	Good (fusion steps ensure consistency)
Development Complexity	Moderate (centralized processing logic)	High (requires distributed algorithms)	Moderate (hybrid model with coordinated fusion)

1.3 Introduction to ROS 2

An open-source middleware platform for robotics applications called ROS was made available to the general public in 2007. It has protocols, libraries, and tools that make building intricate robotic platforms easier. The distributed computing concept upon which ROS is based revolves around nodes, topics, and messages. Individual processes known as nodes carry out calculations and are able to exchange information with one another on various subjects. Nodes exchange messages that are typed data structures over topics called buses. Because of its great degree of modularity, this architecture type makes it possible to construct complicated systems using reused parts.

For code organization and sharing, ROS offers extensive package management. The foundation of software organization is ROS packages. The nodes, libraries, datasets, and configuration files needed to carry out a particular set of tasks are contained in packages. It is simple to share and reuse these packages among various robotic systems. This shortens development time and results in code modularity. It was a first for robotics research when standardized libraries, protocols, and the package system were introduced in ROS. For

academics and developers to exchange and expand on existing work, ROS offers a shared platform. This enables code reuse and quick prototyping, which has led to ROS being a standard in robotics research and development. A sizable and vibrant community actively supports ROS's growth and development.

1.3.1 ROS 2: Next Generation ROS

Introduced in 2017, ROS 2 represented a major development from its predecessor. Figure 1.3 shows how ROS 2 differs from ROS and emphasizes the main enhancements in the more recent version. It demonstrates how ROS 2 removes the application layer centralized Master node found in ROS 1. This supports a system architecture that is more robust and distributed.

Since the introduction of the Data Distribution Service (DDS), the middleware layer has also undergone some changes in the most recent version. The communication protocol was standardized by the DDS, improving performance in real time. Windows, macOS, and real-time operating systems are now compatible with ROS 2 at the OS layer.

Figure 1.3 further displays other ROS 2 features. Two notable additions are better Quality of Service and increased security via the DDS's integrated security. Other enhancements include multi-robot coordination and support for small embedded devices. The changing needs of the robotics community are met by these advancements. The middleware layer has been illustrated through the use of the DDS. Numerous ROS restrictions have been effectively resolved by ROS 2, especially with regard to scalability and real-time performance. ROS 2 provides a more modular and scalable architecture compared to ROS 1, eliminating the need for a centralized master node, as stated in the ROS 2 Documentation[1].

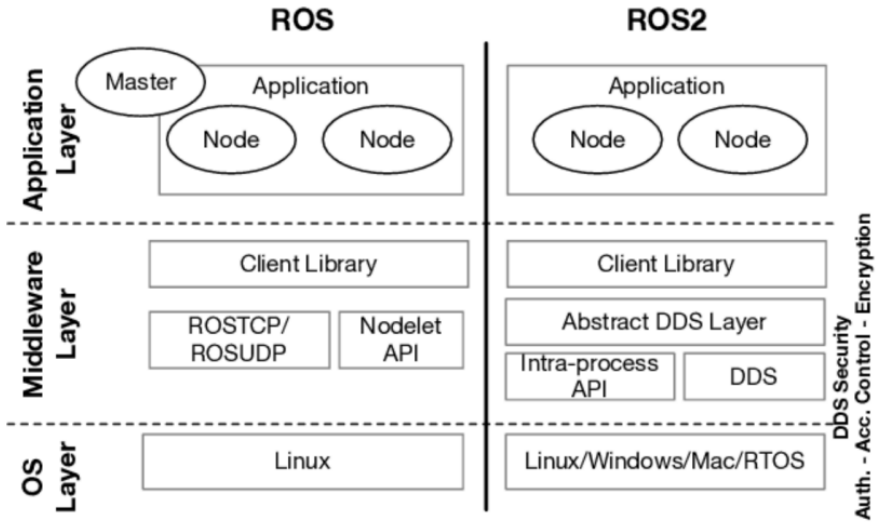


Figure 1.3: ROS/ROS2 architecture overview

[1]https://docs.ros.org/en/humble/

1.3.2 SLAM in ROS and ROS 2

Packages and tools for SLAM algorithm implementation are available in both ROS and ROS 2. ROS 2's enhancements open up a lot of new opportunities for SLAM implementations. gmapping (ROS 1), cartographer (ROS 1 and ROS 2), rtabmap (ROS 1 and ROS 2), and slam toolbox (ROS 2) are a few of the well-known SLAM tools in the ROS ecosystem. One continuing work is the integration of SLAM techniques with ROS 2. Existing SLAM packages are now being converted to ROS 2, while new ones that utilize the enhanced features of ROS 2 are being created ...new ones that utilize the enhanced features of ROS 2 are being created [9].

Conclusion

The purpose, research context, goals, and technique of the R2C-SLAM project were all thoroughly covered in this chapter. The project's goal is to create a reliable, real-time, multi-robot mapping system that is both affordable and flexible by utilizing collaborative SLAM techniques and contemporary robotic development methodologies. The system architecture, implementation, and assessment will be covered in more details in the upcoming parts.

Chapter 2

Requirement Analysis and Structual Design of R2C-SLAM

Introduction

This chapter identifies the R2C-SLAM system's stakeholders and lays out the functional and non-functional expectations in order to describe the system's basic requirements. The R2C-SLAM system's structural design is covered next. It describes the interactions between several modules, including data fusion, communication, and SLAM processing, among multiple robots. It also describes the interaction dynamics and coordination logic that underpin cooperative behavior.

2.1 Requirements Analysis

2.1.1 Stakeholder Identification

The key stakeholders involved in the R2C-SLAM project include:

- **Developers:** Responsible for designing, implementing, testing, and maintaining the system.
- **Academic Supervisor:** Provides technical guidance and oversees the project methodology and progress.
- **End Users:** Individuals or institutions aiming to deploy the collaborative SLAM system in real-world environments, such as educational, industrial, or research applications.

2.1.2 Functional Requirements

The following list outlines the key functional requirements that the R2C-SLAM system must fulfill in order to achieve its intended objectives:

- Real-time SLAM capabilities for each robot.
- Inter-robot communication for map sharing and coordination.
- Autonomous navigation and obstacle avoidance.
- Dynamic task distribution among robots.
- Real-time visualization of the environment and robot locations.
- Simulation support through Gazebo prior to real deployment.

2.1.3 Non-Functional Requirements

In addition to the functional aspects, the R2C-SLAM system must also meet several non-functional requirements to ensure reliability, scalability, and security in real-world deployment:

- **Performance:** Quick reaction time for path planning and localization.
- **Reliability:** Robust communication systems and fault recovery in case of emergencies.
- **Security:** Secure data exchange between agents.
- **Scalability:** Ability to add more robots without significant performance degradation.

2.1.4 Domain Requirements

The domain requirements define the technical and environmental constraints specific to the collaborative SLAM system and its intended operating context:

- Integration with ROS 2 for distributed robotic control.
- Use of standard SLAM algorithms such as ORB-SLAM3 and Cartographer.
- Real-time sensor processing (e.g., LiDAR, RGB-D).
- Communication over the DDS protocol for scalability and reliability.

2.2 Structural Design of R2C-SLAM

Two mobile robots with sensors and actuators and onboard computing modules (e.g. Raspberry Pi) make up the R2C-SLAM architecture, as depicted in Figure 2.1. A ROS 2 DDS-based middleware facilitates both intra-robot communication between nodes within the same robot and inter-robot communication across multiple robots. Each robot operates its own local SLAM algorithm while sharing necessary data with others. The system is modular, with separate ROS nodes handling mapping, location, task coordination, and display. Resolving map overlaps and guaranteeing constant frame references are made easier with central coordination.

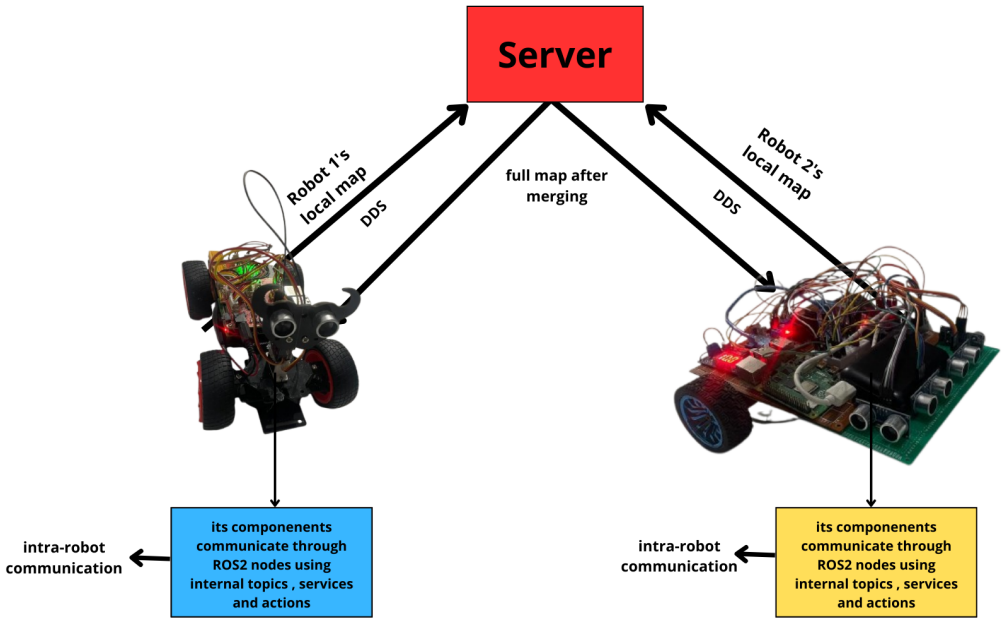


Figure 2.1: R2C-SLAM system architecture

2.2.1 Robot-to-Robot Communication Model

The Data Distribution Service (DDS) is a middleware communication protocol and API standard defined by the Object Management Group (OMG) for real-time, scalable, and high-performance data exchange. It follows a decentralized, peer-to-peer publish-subscribe model and supports various Quality of Service (QoS) policies to ensure reliable communication in distributed systems. In ROS 2, DDS is the default communication layer, enabling seamless data sharing between nodes without needing a central broker.DDS is the default communication middleware in ROS 2, offering real-time and scalable data exchange capabilities suitable for robotic applications [14].

The DDS protocol in ROS 2 is used to implement communication between the robots. Fault tolerance, scalability, and real-time data exchange are guaranteed by this decentralized communication. To synchronize mapping data and exchange important information including robot postures, task status, and local maps, the system uses topic publish-

ing/subscribing and service calls. Quality of Service (QoS) policies are set up to guarantee delivery under network restrictions and provide priority to important messages.

2.2.2 Data Exchange and Coordination Flow

The robots exchange various types of data:

- SLAM updates: local occupancy maps or keyframes.
- Pose and transform data: shared via the TF2 system.
- Exploration status: frontier progress, task assignments.

The coordination layer monitors progress, assigns exploration goals to prevent overlap, and merges maps either on-demand or periodically. Transform buffers and timestamped messages are used to control synchronization. In this, the comparative analysis of collaborative architectures, previously discussed in section 1.2.3, led to the choice of a semi-centralized coordination approach for R2C-SLAM system, as it strikes a balance between resilience and efficiency.

2.2.3 System Behavior Specification

Sequence diagrams that show the flow of operations during the exploration and map merging stages are used to model the behavior of the system.

Exploration Use Case Sequence Diagram

The robots launch SLAM nodes, localize themselves, navigate the area, receive task assignments from the coordinator, and exchange mapping data during this sequence. The diagram shows how messages are sent for task completion, collision avoidance, and pose updates. Figure 2.2

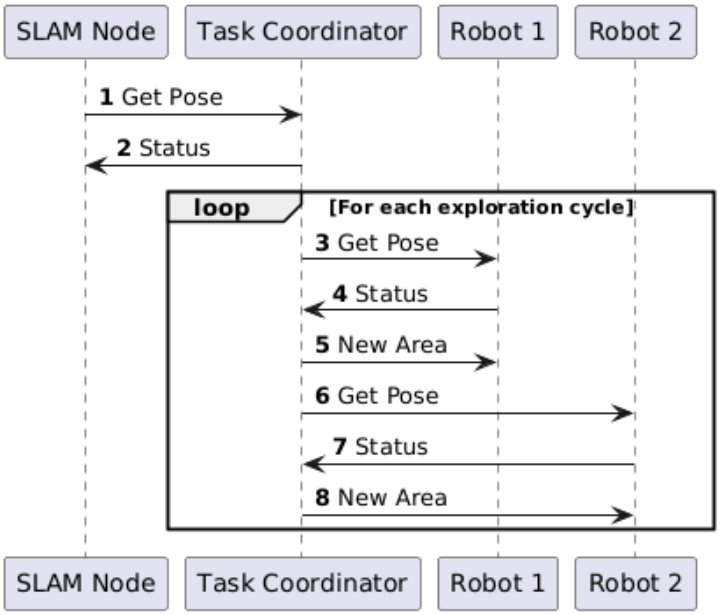


Figure 2.2: Sequence diagram: exploration use case

Map Merging Use Case Sequence Diagram

This procedure is started on a regular basis or when a loop closure is detected. Robots align their maps via scan matching or graph optimization after sending their map segments to the central node or straight to one another. A fused map is the end product, and both agents get a broadcast of it for updated navigation.

Conclusion

The requirements for the R2C-SLAM project outlined in this chapter form the cornerstone for all subsequent design, development, and testing efforts, ensuring the system meets stakeholder needs and performs effectively in real-world scenarios.

Chapter 3

R2C-SLAM Simulation

Introduction

The development of autonomous mobile robots depends heavily on simulation, which provides a flexible and safe setting for testing algorithms prior to their practical implementation. In order to assess SLAM algorithms, inter-robot communication, map merging, and user interfaces, we provide a comprehensive simulation setup in this chapter that resembles, as much as possible, actual testing situations. The purpose of the simulation is to use ROS 2 and Gazebo to assess the system's performance in dynamic yet controlled conditions.

3.1 Robot Choice

TurtleBot3 is a low-cost, fully open-source, programmable mobile robot platform designed by ROBOTIS in collaboration with the Open Source Robotics Foundation (OSRF) [6]. It is specifically built for learning, research, and rapid prototyping in robotics and is fully compatible with ROS and ROS 2. The modular design of TurtleBot3 supports a wide range of sensors and actuators, making it a popular choice for SLAM, navigation, and multi-robot experiments. For simulation purposes, and due to its wide adoption, we will use the TurtleBot3 platform, which we introduced in Subsection 1.2.1.

TurtleBot3 is equipped with a set of essential sensors and actuators for autonomous navigation and perception, as illustrated in Figure 3.1. Precisely, it includes a 2D **LiDAR** (Light Detection and Ranging) (e.g., LDS-01) for obstacle detection and SLAM, an **IMU** (Inertial Measurement Unit) for orientation and motion tracking, and **wheel encoders** for odometry.

LiDAR is a remote sensing technology that measures distances by illuminating the target with laser light and measuring the reflection time, providing accurate spatial information for mapping and navigation [8]. Figure 3.2An IMU is a sensor that combines accelerometers, gyroscopes, and sometimes magnetometers to measure linear acceleration

and angular velocity, enabling precise tracking of the robot's orientation and movement [15]. Wheel encoders are sensors attached to a robot's wheels that measure the rotation of the wheels to estimate movement. They work by detecting changes in position—either using optical, magnetic, or mechanical methods—and convert wheel rotation into digital signals.

Furthermore, TurtleBot3 uses two servo motors for differential drive, allowing precise control of movement and rotation.

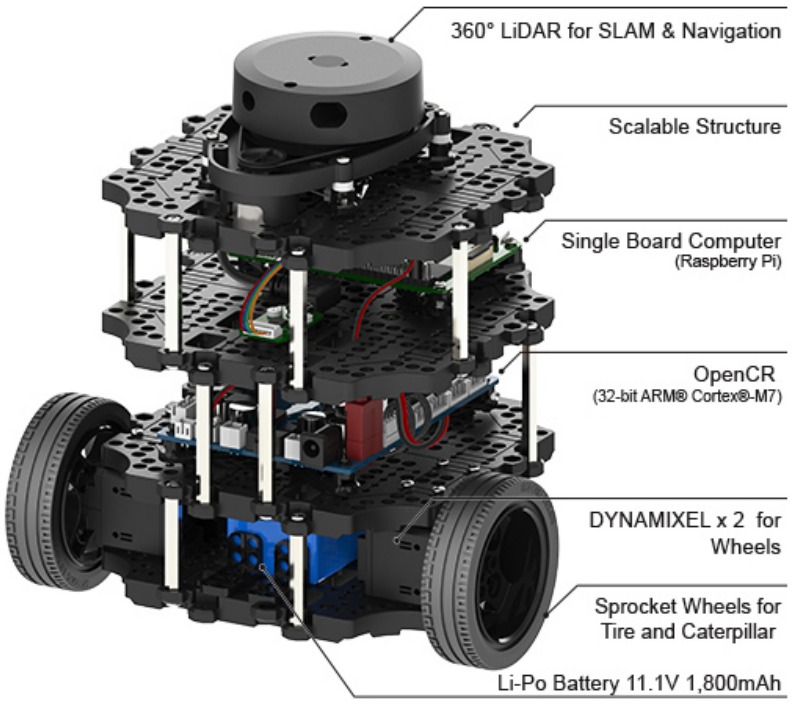


Figure 3.1: TurtleBot3 Burger components description. [Source: `https://robotis.co.uk/turtle-burg.html`]

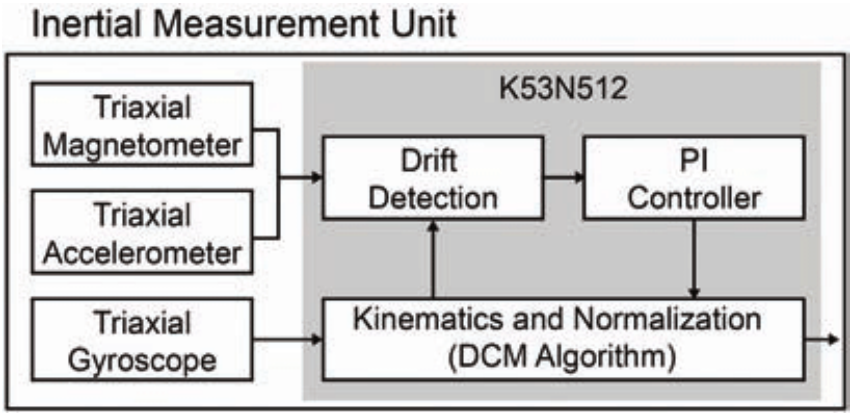


Figure 3.2: A labeled diagram of an IMU showing its internal components [Source: [10]]

3.2 Software Stack and Simulation Environment

The software stack is built upon widely adopted technologies in the robotics community. At its core, it leverages ROS 2 Humble as the primary framework, providing support for SLAM algorithm integration, node orchestration, and inter-process communication. Two key SLAM algorithms are integrated: `GMapping`, known for its efficiency in 2D mapping, and `Cartographer`, which supports real-time mapping. Development is carried out using a combination of Python and C/C++.

In order to replicate the setup of a `Turtle_Bot3`-like robot with a 2D-Lidar sensor, the simulation environment was created using *Gazebo* (Figure 3.3) connected with ROS 2, with the aforementioned software setup. The workstation running Ubuntu 22.04 with ROS 2 Humble, together with necessary packages like `slam_toolbox`, `nav2bringup`, and `turtlebot3_simulations`, is used to execute the simulation. In order to evaluate autonomous navigation and mapping skills, the development setup enables us to generate a variety of virtual environments, such as rooms and corridors. `Rviz_2` (Figure 3.4) is widely used for visualization, allowing for pose estimation, map building, and real-time tracking of the robot's sensor data.

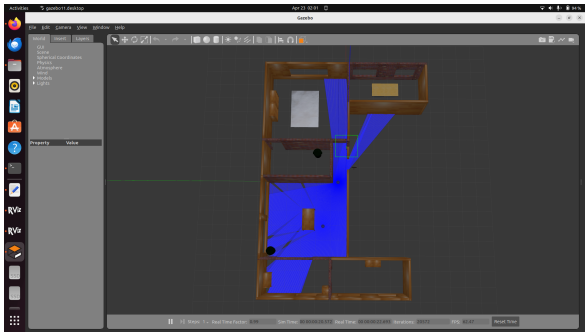


Figure 3.3: Gazebo Environment

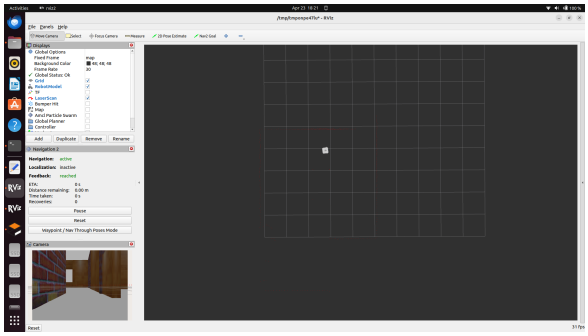


Figure 3.4: Rviz Visualization

3.3 Intra-Robot SLAM Integration

The `slam_toolbox` package, which enables real-time mapping and localization, is used in the simulation to implement SLAM. As it moves across a Gazebo world, the robot gathers laser-scan data to gradually create a map. We examined the correctness and comprehensiveness of the produced maps in order to assess the performance of many SLAM algorithms, such as `Cartographer` and `ORB_SLAM3`. A stable and expandable system design is made possible by the ROS 2 middleware, which is built on DDS and manages communication between nodes. Sensor data and coordinate frames are transmitted using topics like `/scan`, `/odom`, `/tf`, and `/map`. Custom launch files that guarantee correct startup and topic remapping are used to launch the SLAM nodes.

3.4 Inter-Robot Communication

Two robots were created in the same Gazebo world utilizing namespace isolation in order to mimic inter-robot communication. The DDS-based communication mechanism in ROS 2 enables effective message transfer between these isolated nodes, and each robot functions under its own namespace, publishing topics such as `/robot1/scan` and `/robot2/odom`. To guarantee smooth data flow, the communication module was tested with tools like `rqt_graph` and `ros2 topic echo`. With the help of this simulation system, we were able to evaluate how well communication tactics operate in different network scenarios and spot possible synchronization or bottleneck problems.

3.5 Map Merging in Multi-Robot Simulation

3.5.1 Overview and System Architecture

Map merging is an essential feature of any collaborative SLAM system, allowing multiple robots to combine their locally generated maps into a unified global representation. This capability in the R2C-SLAM system enables autonomous exploration by each robot while contributing to a shared understanding of the environment.

To implement this, we used the open-source, ROS 2 package `mapMergeForMultiRobotMapping-ROS2`, specifically designed for multi-robot occupancy grid merging. This package supports distributed SLAM by subscribing to the map topics of different robots and merging their occupancy grids based on relative transformations [5].

Each robot publishes its map (`/robotX/map`) and transform frames (`/tf`, `/tf_static`) under a unique namespace. The map merging node listens to these topics and projects each local map into a global coordinate frame (typically `map`) using transformation data provided by ROS 2's `tf2` system. The resulting unified map is published as `/map_merged` and visualized in RViz2 for real-time monitoring.

The system relies on several architectural principles:

- **Transform Synchronization:** Robots must publish their `tf` data consistently and accurately. Any delay or inconsistency in transform broadcasting can cause misalignment in the merged output.
- **Namespace Isolation:** Each robot operates in an isolated namespace (e.g., `/robot1`, `/robot2`) to prevent topic collisions and ensure modularity.
- **Occupancy Grid Fusion:** The package overlays each map using its transformation and resolves overlaps by prioritizing the most recent or confident grid cells.

3.5.2 Implementation Challenges

Despite the controlled nature of the simulation, the map merging process encountered several technical challenges. One significant issue was **alignment error**. When initial poses or `tf` transforms were not properly configured, the merged map displayed duplicated walls, skewed hallways, or broken continuity in overlapping regions. This was mitigated through careful manual initialization and repeated calibration.

Another challenge involved **map resolution mismatches**. Even minor differences in SLAM configuration between robots produced stitching artifacts. To address this, all robots were set up with identical parameters, including map resolution, scan range, and update frequency.

Communication latency also affected performance. While ROS 2 DDS handles real-time messaging efficiently, occasional **synchronization lags** were observed. The map merging node was tuned to buffer `tf` data and account for timing discrepancies, which helped stabilize the merged output.

The package required **fine-tuning of various parameters** such as `merging_rate`, `frame_id`, and robot initial offsets. These were optimized on the basis of iterative testing to achieve responsive and accurate map integration.

Overall, the simulation validated the reliability of ROS 2 package `mapMergeForMultiRobotMapping-ROS2` and highlighted key factors to consider for real-world deployment. The successful implementation demonstrates the package's potential to support robust and scalable collaborative SLAM.

3.5.3 Visualization and Monitoring Tools

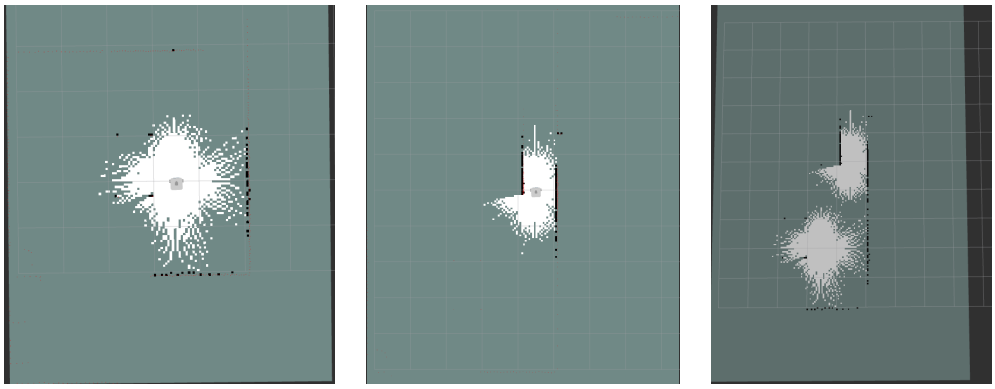
For the simulation to be monitored and controlled, user interfaces are crucial. The main visualization tool for this research is `Rviz2`, which displays the robot's location, the map created by SLAM, and its navigation routes. The `teleop_twist_keyboard` package is used to implement keyboard teleoperation, enabling manual control during testing. Furthermore, the communication graph between nodes is visualized using `rqt_graph`, and real-time SLAM parameter adjustment is facilitated by `rqt_reconfigure`. These tools provide valuable insights into the internal states of the system and support rapid troubleshooting and optimization.

3.6 Map Merging Results

To better illustrate the evolution of map merging in our multi-robot SLAM simulation, we capture and compare the occupancy grid maps generated before and after robot movement. These visualizations help assess the accuracy and efficiency of the merging process over time.

3.6.1 Pre-Movement Map Merging Results

At the initial stage, the two robots start exploring from different positions without yet moving significantly. The local maps are generated independently, and the map merging node combines them using the initial `tf` transformations. This allows us to verify the correctness of the merging setup and robot initialization, as illustrated in Figure 3.5.

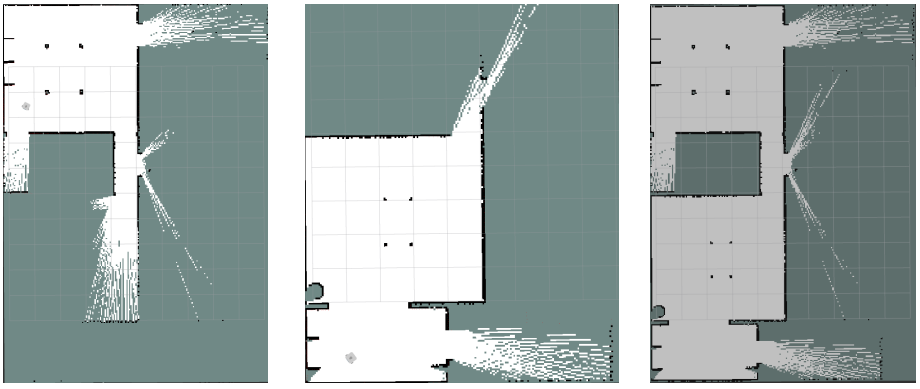


(a) Map of Robot 1. (b) Map of Robot 2. (c) Merged Map.

Figure 3.5: Pre-Movement Map Merging Results.

3.6.2 Incremental Map Merging During Robot Movement

After the robots have navigated and explored more of the environment, each robot updates its local occupancy grid. The map merging node continuously combines these updated maps into a unified global map. This stage reflects the real collaborative SLAM performance under motion and helps evaluate merging robustness, as illustrated in Figure 3.6.



(a) Map of Robot 1. (b) Map of Robot 2. (c) Merged Map.

Figure 3.6: Incremental Map Merging Results.

Conclusion

The simulation environment created for this research provides a thorough testing ground

for assessing mobile robots' capacity for autonomous navigation. We have developed a strong simulation pipeline that replicates real-world settings by integrating SLAM, inter-robot communication, map merging, and user interface tools. Higher dependability and performance in real-world settings are ensured by the lessons learnt from this simulation phase, which greatly aids in system refinement prior to deployment.

Chapter 4

R2C-SLAM Real-world Deployment

Introduction

A particular hardware and software environment was established in order to carry out the R2C-SLAM project. In order to design a collaborative SLAM solution between two mobile robots while adhering to cost, performance, and reliability restrictions, this setting was chosen. Our SLAM-based system's physical hardware implementation is covered in this chapter, along with the setup, algorithm deployment, robot-to-robot communication, and real-time monitoring tools.

4.1 Hardware Setup of Both Robots

Our project is based on collaboration between two robots. Due to availability and cost constraints, the robots used are not identical. However, we see this as an opportunity to test and evaluate interoperability between heterogeneous systems.

We begin by detailing the hardware setup of each robot individually, followed by the software setup, which is functionally identical at an abstract level.

4.1.1 R2C-R1 Hardware Setup

As shown in Figure 4.1, the R2C-R1 robot is built using the SunFounder PiCar-S Smart Car Kit[1], a platform designed for the Raspberry Pi.

[1] `https://docs.sunfounder.com/projects/picar-s/en/latest/`

Figure 4.1: Illustration of Robot R2C-R1

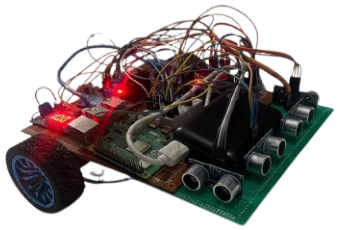


Figure 4.2: Illustration of Robot R2C-R2

4.1.2 R2C-R2 Hardware Setup

The R2C-R2 robot (Figure 4.2) has been entirely designed and developed in-house! The hardware architecture detailed hereafter is illustrated in Figure 4.5.

Processing: A Raspberry Pi 4 serves as the primary processing unit in each robot. It manages the ROS 2 nodes in charge of sensor fusion, SLAM, and general coordination duties. The Arduino Mega microcontroller, which controls the low-level control of the robot's actuators, is connected to the Raspberry Pi 4 via a wired serial connection. The Raspberry Pi may concentrate on higher-level decision-making and map creation while the Arduino manages real-time motor controls and H-Bridge connectivity.

Perception: The robot perceives its environment using: (i) **Four ultrasonic sensors** (Figure 4.3) placed around the chassis for obstacle detection and distance measurements, and (ii) an external **IMU sensor** (Inertial Measurement Unit)(Figure 4.4) that provides orientation and acceleration data for improved localization and motion tracking.

Note that ultrasonic sensors [2] are devices that measure distance by emitting high-frequency sound waves and calculating the time it takes for the waves to reflect off objects and return to the sensor. LiDAR sensors are not used. Rather, the robot's position is estimated and an environment map is created by fusing data from the IMU and ultrasonic sensing.

Power and Communication: A portable battery pack powers the entire setup, connecting to the Raspberry Pi 4 and Arduino Mega via a voltage regulator to provide safe and consistent power levels. Since all component communication is done locally over serial connections rather than through Wi-Fi or Bluetooth modules, there is less interference, less energy usage, and more dependability on longer missions.

Communication Modules: The robots communicate wirelessly via **Wi-Fi**, using the **ROS 2 DDS** protocol for inter-robot data exchange. DDS (Data Distribution Service), the default middleware in ROS 2, supports decentralized, real-time communication between distributed nodes. To avoid topic collisions and ensure scalability, each robot operates within its own namespace. Inter-robot cooperation is enabled through the exchange of topics such as `/robot1/scan` and `/robot2/odom` across the network. The communi-

cation system is evaluated in terms of message reliability, latency, and bandwidth usage, especially under conditions involving multiple concurrently operating robots.

Figure 4.3: (i)Ultrasonic sensors.

Figure 4.4: (ii)External IMU module used for orientation and acceleration sensing.

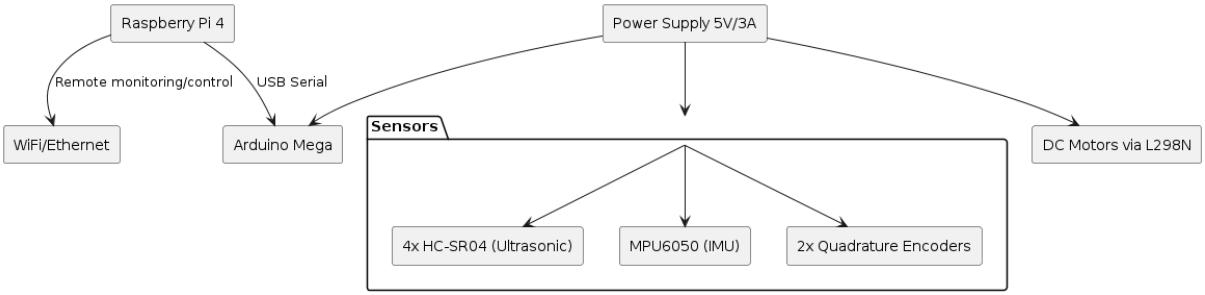


Figure 4.5: R2C-R2 Hardware Architecture Diagram

4.2 Software Stack in Real Deployment

This system's software modules are all created using a modular ROS 2-based codebase. The project's GitHub repository contains the complete implementation. Key files include:

- `slam/launch/slam_launch.py`: for SLAM integration.
- `hardware_control/ultrasonic_reader.py`: for ultrasonic sensor readings.
- `motors/back_wheels.py`: for motor control.
- `communication/robot_comm.py`: for inter-robot message exchange. These components were integrated into the launch file `launch/real_robot.launch.py` to coordinate SLAM, navigation, and communication.

With modifications for hardware interface, the software stack replicates the simulation environment. Along with required packages like `slam_toolbox`, `nav2_bringup`, and

`robot_localization`, the Raspberry Pi 4 has ROS 2 Humble installed. Real-time sensor data is provided by integrated hardware drivers for the Lidar, motor encoders, and IMU. Launch files are altered to keep all modules in sync and to account for the actual sensor topics. Figure 4.6

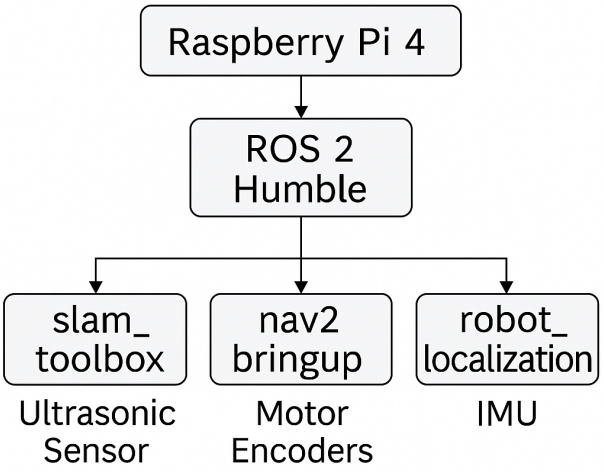


Figure 4.6: Software Stack in Real Deployment

4.2.1 In-Robot SLAM Deployment

In live mode, `slam_toolbox` is used to perform SLAM. The robots use actual sensor data to dynamically create a map as they move through an indoor environment on their own. In contrast to the simulation, real-world deployment has to deal with localization drift and incomplete data. To maximize accuracy, extensive testing is done to adjust SLAM parameters such as resolution, scan matcher settings, and loop closure thresholds. The produced maps are stored so they can be examined and contrasted with existing floor layouts.

Because LiDAR sensors were not used in our setup, we developed an alternative method using four ultrasonic sensors positioned on the front, left, and right sides of the robot. A custom ROS 2 node named `ultrasonic_to_laserscan` was implemented to convert ultrasonic readings into `sensor_msgs/LaserScan` messages. This allowed the system to emulate LiDAR-like behavior and maintain compatibility with SLAM tools like `slam_toolbox`. The generated scan data was then fused with IMU and encoder information to improve localization accuracy. The complete data flow for this sensor fusion process is shown in Figure 4.7. This allowed the system to emulate LiDAR-like behavior and maintain compatibility with SLAM tools like `slam_toolbox`. The generated scan data was then fused with IMU and encoder information to improve localization accuracy. The complete data flow for this sensor fusion process is shown in Figure 4.7.

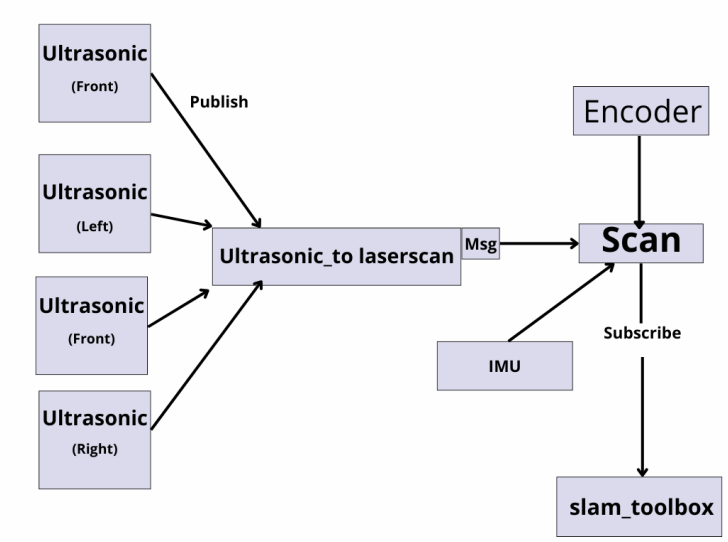


Figure 4.7: Architecture of ultrasonic-based LiDAR simulation using four ultrasonic sensors, an IMU, and encoders (mounted on R2C-R2).

To achieve robust localization, sensor fusion was implemented using the `robot_localization` package provided in ROS 2. Specifically, we used the `ekf_node` to fuse data from three sources: the IMU, the encoders (providing odometry), and the laser-like data synthesized from the ultrasonic sensors. The ultrasonic readings were first converted into `LaserScan` messages using a custom node, and then integrated as range measurements in the SLAM process. The `ekf_node` was configured to use the IMU's orientation and angular velocity, along with wheel encoder-derived linear velocity, to estimate the robot's pose in real time. Careful tuning of the covariance matrices and update rates was performed to mitigate the impact of ultrasonic noise and ensure stability during motion. This sensor fusion pipeline was deployed identically on both robots, though the quality of input data slightly differed due to hardware differences between R2C-R1 and R2C-R2.

For the R2C-R1 robot, built on the SunFounder PiCar-S platform, SLAM deployment required adapting to its specific hardware constraints. Unlike R2C-R2, which features multiple ultrasonic sensors, R2C-R1 is equipped with a single ultrasonic sensor mounted on a servo motor, allowing it to scan the environment by rotating the sensor. To emulate LiDAR-like behavior, a custom ROS 2 node was developed to synchronize distance readings with servo angle positions, generating `sensor_msgs/LaserScan` messages from the single-point measurements. This allowed seamless integration with `slam_toolbox` for 2D SLAM. Orientation data was obtained from the onboard IMU, and sensor fusion was carried out using the `robot_localization` package configured with an Extended Kalman Filter (EKF). Compared to R2C-R2, the R2C-R1 setup produces sparser and slower scan data due to the mechanical rotation delay, which results in slightly lower mapping accuracy and increased susceptibility to localization drift. Nevertheless, the robot can successfully contribute to the collaborative SLAM system and participate in the map merging process, demonstrating interoperability across heterogeneous platforms.

4.3 Real-World Map Merging

`Rviz2` operating on a central workstation facilitates field monitoring and control. Map updates, robot trajectories, and overlays of sensor data are examples of real-time visual feedback. Field operators can use `teleop_twist_keyboard` to manually operate the robots or send navigation goals. During deployment, diagnostic tools such as `rqt_graph` and `rqt_reconfigure` are used to track node interactions and make dynamic parameter adjustments.

4.4 Current State of Development

As of May 2025, we successfully completed the entire simulation phase of the R2C-SLAM system. Both robots were simulated in Gazebo, and all core functionalities were validated, including SLAM, inter-robot communication, and map merging using the `mapMergeForMultiRobotMapping-ROS2` package.

In the real-world deployment, we managed to get both robots—R2C-R1 and R2C-R2—operating independently. Each robot was able to run its SLAM node using onboard sensors and transmit sensor data over the ROS 2 DDS communication layer. The data conversion pipeline from ultrasonic sensors to `LaserScan` messages functioned as expected, and real-time localization was achieved using the `robot_localization` package. However, due to time constraints, we were not able to obtain any local maps from the robots. As a result, map merging could not be attempted or validated in the real environment.

This limitation is currently being addressed, and we plan to complete the mapping and merging steps in upcoming tests. Overall, the system demonstrates stable performance for individual SLAM and communication tasks in simulation, setting a strong foundation for full collaborative SLAM functionality in the near future.

Conclusion

The practical viability of our solution is confirmed by the real-world implementation. The SLAM-based method effectively permits autonomous exploration and mapping in spite of obstacles including irregular sensor input and communication unpredictability. Field testing, communication design, and hardware integration experience all provide important insights for creating scalable, reliable multi-robot systems.

Chapter 5

Simulation vs Real-World Deployment: Comparison and Lessons Learned

Introduction

While real-world implementation results in physical constraints, capture errors, and unpredictable environments, simulation offers a quiet, controlled environment for validating SLAM strategies. This chapter establishes a comparison between these two R2C-SLAM test phases, while emphasizing the limitations of both, as well as the lessons learned.

5.1 Issues of the Sensing Technology

5.1.1 Comparison

The system perceives using a simulated 2D LiDAR sensor in simulation. The real-world system, on the other hand, uses ultrasonic sensors, which have a much smaller field of vision and range and are much louder. The quality of obstacle identification and environment mapping is impacted by these variations.

Ultrasonic sensors are more affordable than LIDAR, but they have a lesser resolution, which makes them appropriate for some low-cost robotic applications [4]. Table 5.1 compares both sensing technologies.

Feature	**Ultrasonic**	**LIDAR**
Cost	Low	High
Size	Small	Medium-Large
Speed Detection	Low	Medium
Sensitivity to color	No	No
Robust to weather	High	Medium
Robust to day and night	High	High
Resolution	Low	High
Range	Short	Long

Table 5.1: Comparison between Ultrasonic and LiDAR sensors for perception.

5.1.2 Lessons Learned

The utilization of ultrasonic sensors posed some challenges. These sensors are of low cost. However, they are known to produce quite noisy as well as unreliable data, especially when interacting with such soft materials, narrow objects, or angled surfaces. Ultrasonic sensors, both directional and suffering from multi-path reflection effects given a limited field of view, stand unlike LiDAR, which provides accurate, dense, wide-angle range data overall. This led to:

- Inconsistent obstacle detection
- Phantom readings (ghost echoes)
- Dropouts in open space

These artifacts led to pose drift, map distortion, and a lot of incorrect loop closures in SLAM.

5.2 Issues with Mapping Accuracy

Maps created in virtual LiDAR simulation environments are often continuous, smooth, and geometrically accurate. This is because of optimal circumstances, such as flawless sensor models, noise-free odometry, and extremely predictable robot behavior. These environments produce maps that are excellent for evaluating SLAM algorithms because they usually have crisp edges, well-defined walls, and constant scaling.

However, using ultrasonic sensors in real-world situations presents a number of difficulties. As highlighted in Table 5.2, these sensors are sensitive to a range of environmental conditions and are more likely to be affected by noise. The accuracy of the distance readings can be affected by the air temperature, humidity, angle of incidence, and surface reflectance.

Furthermore, compared to LiDAR, ultrasonic sensors often have a far smaller field of vision, which results in more blind patches and sparse environmental sampling.

Parameter	**Simulation (LiDAR)**	**Real World (Ultrasonic)**
Scan Matcher Resolution	0.05 m	0.15 m
Max Range	3.5 m	1.5 m
Loop Closure Enabled	Yes	Partial
Update Frequency	10 Hz	2–3 Hz

Table 5.2: Comparison of key SLAM parameters between simulation and real-world deployment.

We *anticipate* that because of the intrinsic inaccuracy and sparsity of the created maps, it will be challenging to merge maps from several ultrasonic-based SLAM outputs. Ultrasonic maps often display gaps, misaligned walls, and fragmented features, in contrast to simulated LiDAR environments where local maps align nearly perfectly. We would rather require:

- Manual adjustment of initial transform offsets.
- Pose correction during merging .
- Additional filtering of maps before merging .

5.3 Issues with Communication and Synchronization

In simulation, communication reliability was higher. Intermittent message loss between robots occurred in the field due to Wi-Fi interference and fluctuating signal strength. In order to reduce packet loss and give priority to synchronization messages, DDS Quality of Service settings were adjusted.

Despite its convenience, Wi-Fi-based communication was extremely sensitive to environmental elements including metal surfaces, walls, and robot distance from one another. In numerous indoor real-world situations, this resulted in:

- Delayed message delivery.
- Dropped synchronization signals.
- Incomplete or out-of-sync maps.

When robots operated in signal-shadowed areas or were widely separated, this was particularly difficult. Although this was lessened by DDS Quality of Service settings, real-time coordination continued to occasionally suffer.

5.4 Issues with Constrained Resources

5.4.1 Observation

The Raspberry Pi 4 Model B boards utilized in the system have CPU and memory limits during extended missions, despite their strength for embedded applications. Map merging, running multiple SLAM threads, and GUI visualization (like RViz2) frequently resulted in:

- High CPU usage (>80%).
- Thermal throttling (temperatures >70°C).
- Swap memory usage leading to system slowdowns.

5.4.2 Lessons Learned

To lessen the stress on the Pi, tasks like real-time parameter adjustment and live visualization have to be transferred to a distant workstation. We were able to offload portions of the processing to remote workstations by using scripts found in our GitHub source to handle CPU-intensive operations like SLAM processing and real-time visualization (e.g., RViz2).

Conclusion

Important distinctions are found when comparing simulation and real-world deployment in R2C-SLAM. Virtual LiDAR simulation offers precise, continuous maps with no drift, which makes it perfect for preliminary algorithm validation. However, issues like noise, short range, and localization drift make real-world deployment of ultrasonic sensors difficult and lead to fragmented maps. Stability of Wi-Fi communication also becomes a problem. In order to close this gap, sensor data must be filtered, SLAM settings adjusted, and any hardware upgrades taken into account. To create a dependable multi-robot SLAM system, simulated and real-world testing must be combined.

Conclusion and Outlook

Using two inexpensive mobile robots, we designed and implemented **R2C-SLAM**, a collaborative SLAM system. We accomplished autonomous navigation, inter-robot coordination, and real-time map building in both simulated and real-world settings by utilizing the ROS 2 framework and DDS-based communication protocols.

In order to guarantee reliable operation under hardware and communication limitations, the system architecture combined sensor fusion, map merging, and centralized control. We demonstrated the viability of multi-robot SLAM in resource-constrained environments by validating the system with comprehensive simulations in Gazebo and real-world tests utilizing a Raspberry Pi 4 and ultrasonic sensors.

The project also included analyzing performance under simulated and real-world settings and creating a user interface for monitoring and control. Important restrictions and technological difficulties such sensor noise, communication latency, and map misalignment were noted and thoroughly examined.

Though the present system satisfies its fundamental goals, some improvements could be taken into account for forward development:

- **Sensor Upgrade:** Replace ultrasonic sensors with LiDAR or RGB-D cameras to increase mapping precision and resilience.
- **Algorithmic Optimization:** Integrating advanced SLAM algorithms like RTAB-Map and ORB-SLAM3 with improved loop closure detection and graph optimization.
- **N-Robot Extension:** Expanding the technology to enable more than two robots in large-scale collaborative exploration missions.
- **Autonomous Task Planning:** Using frontier-based exploration algorithms and distributed work assignment to fully automate the exploration process.
- **Robust Communication:** Improving fault tolerance through mesh networking or fallback protocols in the event of Wi-Fi failure.

- **Performance Evaluation:** Introducing quantitative performance indicators and automated benchmarking tools to track SLAM accuracy, resource use, and communication latency.
- **Real-World Applications:** adapting the technology to real-world situations like smart campus mapping, warehouse automation, and catastrophe response.

The foundation for a scalable and reasonably priced collaborative SLAM platform has been established by this effort. R2C-SLAM may develop into a more durable, intelligent, and self-sufficient system that can be used in a variety of robotic applications with additional improvements.

Bibliography

[1] Gene Alvarez. Gartner top 10 strategic technology trends for 2025. `https://www.gartner.com/en/articles/top-technology-trends-2025`, October 2024. Accessed: 2025-05-06.

[2] Alessio Carullo, Marco Parvis, et al. An ultrasonic sensor for distance measurement in automotive applications. *IEEE Sensors journal*, 1(2):143, 2001.

[3] Yucheng Chen, Xiang Wang, Shuangyan Liu, Dingguo Wang, Qingxuan Zhao, Zhaoxiang Li, Chengfei Yu, and Rui Yu. Overview of multi-robot collaborative slam from the perspective of data fusion. *Machines*, 11(6):653, 2023.

[4] A. Falkowski and P. Baranski. A comparison of range sensors for mobile robot localization. In *Proceedings of the International Conference on Automation, Robotics and Applications*, 2017.

[5] Andrei K. and contributors. mapmergeformultirobotmapping-ros2: Multi-robot map merging in ros 2. `https://github.com/MAPIRlab/mapMergeForMultiRobotMapping-ROS2`, 2024. Accessed: 2025-05-07.

[6] HyunMyung Kim, Jinwook Kim, and Bum-Jae You. Turtlebot3: A modular, compact, and open-source mobile robot for education and research. In *2019 International Conference on Advanced Robotics (ICAR)*, pages 597–602, 2019.

[7] Pierre-Yves Lajoie, Benjamin Ramtoula, Fang Wu, and Giovanni Beltrame. Towards collaborative simultaneous localization and mapping: A survey of the current research landscape. *arXiv preprint arXiv:2108.08325*, 2021.

[8] Xiang Liu. A review of lidar technology and its use in remote sensing applications. *Sensors*, 21(10):3427, 2021.

[9] Steven Macenski, Francisco Martín, Ruffin White, and Francisco Ginés Clavero. Slam toolbox: Slam for the dynamic world. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8794–8801. IEEE, 2020.

[10] Maria Martins, Arlindo Elias, Carlos Cifuentes, Manuel Alfonso, Anselmo Frizera, Cristina Santos, and Ramón Ceres. Assessment of walker-assisted gait based on principal component analysis and wireless inertial sensors. *Revista Brasileira de Engenharia Biomédica*, 30:220–231, 2014.

[11] Author Name. Implementation of an autonomous collaborative slam approach. Master's thesis, Politecnico di Torino, 2023.

[12] ROBOTIS. Turtlebot3 slam guide. `https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/`, 2021. Accessed: 2025-04-30.

[13] Sajad Saeedi, Liam Paull, Mae Seto, and Howard Li. Multiple-robot simultaneous localization and mapping: A review. *Journal of Field Robotics*, 33(1):3–46, 2016.

[14] Yadunandana R. V. and Jesse P. V. A performance evaluation of ros2 communication for real-time robotic applications. In *2019 IEEE International Conference on Industrial Technology (ICIT)*, pages 1493–1499, 2019.

[15] Oliver J. Woodman. An introduction to inertial navigation. Technical Report UCAM-CL-TR-696, University of Cambridge, Computer Laboratory, 2007.